Assignment 1: Log Mel Filter Banks & Speech Classification

Часть 1 — Реализация LogMelFilterBanks

Реализован слой `LogMelFilterBanks` на базе `torch.stft` и `torchaudio.functional.melscale_fbanks`. Слой принимает батч аудиосигналов формы `(batch, time)` и возвращает логарифм мел-спектрограммы формы `(batch, n_mels, n_frames)`.

Корректность реализации проверена сравнением с эталонной `torchaudio.transforms.MelSpectrogram`: максимальное абсолютное отклонение составило **9.54e-07**, что соответствует погрешности числа с плавающей точкой и подтверждает идентичность реализаций.

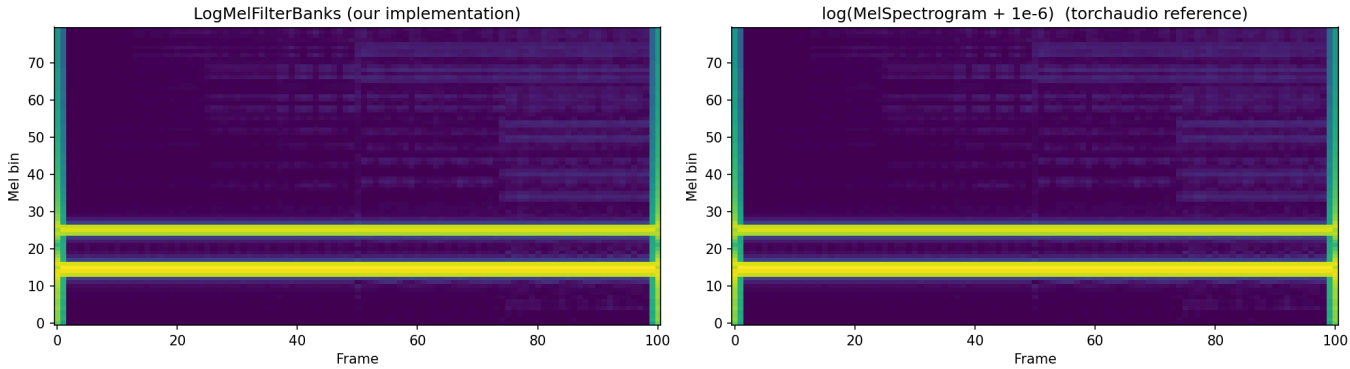

Рис. 1 — Сравнение нашей реализации (слева) и эталонной torchaudio (справа) на синтетическом сигнале.

Часть 2 — Влияние числа мел-фильтров (n_mels)

Проведены эксперименты с `n_mels` ∈ {20, 40, 80} при фиксированном `groups=1`. Задача — бинарная классификация «yes» / «no» на датасете Google Speech Commands.

Для данной задачи увеличение числа мел-фильтров сверх 20 избыточно: различия в итоговой точности между конфигурациями невелики. При `n_mels=20` модель демонстрирует наименьшую ошибку на обучении и более стабильную сходимость — кривая точности на валидации имеет меньше скачков по сравнению с `n_mels=40` и `n_mels=80`. Впрочем, эта нестабильность при больших значениях может быть частично объяснена шумом внутри самого датасета, а не архитектурными причинами.

Таким образом, для простых задач бинарной классификации речи компактное представление (`n_mels=20`) оказывается достаточным и при этом снижает вычислительную нагрузку.

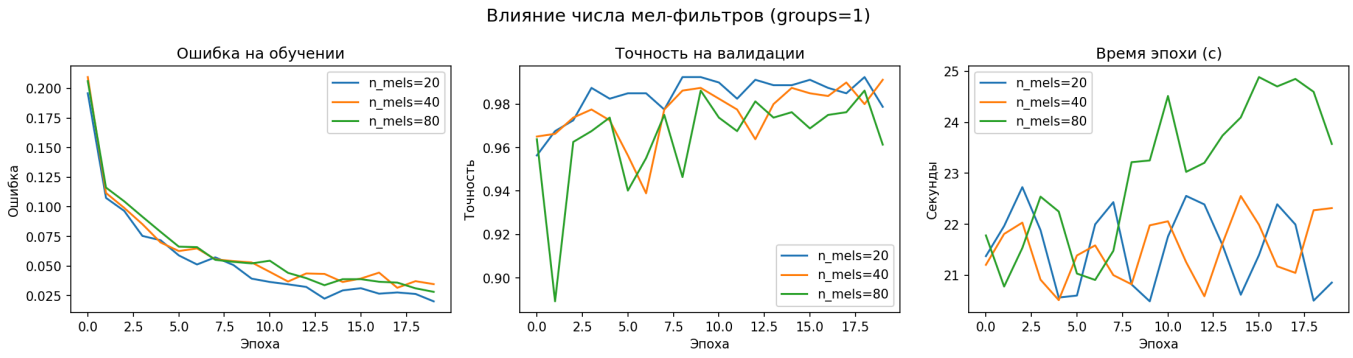

Рис. 2 — Кривые обучения для разных значений n_mels.

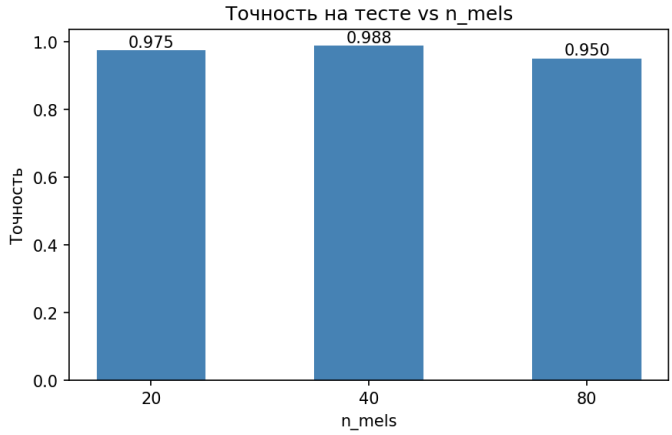

Рис. 3 — Точность на тестовой выборке в зависимости от n_mels.

Часть 3 — Влияние параметра groups в Conv1d

Проведены эксперименты с `groups` ∈ {1, 2, 4, 8, 16} при фиксированном `n_mels=80`.

groups	Параметры	FLOPs (M)	Время эпохи (с)	Точность (test)
1	53 250	4.11	21.40	0.9842
2	27 138	2.09	21.52	0.9357
4	14 082	1.09	21.37	0.9672
8	7 554	0.59	23.85	0.9660
16	4 290	0.34	25.38	0.9284

Таблица 1 — Сравнение конфигураций по параметрам, FLOPs, времени и точности.

С ростом `groups` число параметров и FLOPs backbone снижаются практически линейно: при `groups=16` модель содержит в **12 раз меньше параметров** (4 290 против 53 250) и потребляет в **12 раз меньше FLOPs** (0.34 M против 4.11 M) по сравнению с baseline.

Время эпохи при этом не уменьшается, а наоборот — слегка растёт (с 21.4 до 25.4 с). Это объясняется тем, что групповые свёртки менее эффективно утилизируют GPU: параллелизм внутри каждой группы ниже, и накладные расходы на синхронизацию перевешивают выигрыш от меньшего числа операций.

Точность на тесте остаётся высокой вплоть до `groups=8` (0.966) и заметно падает только при `groups=16` (0.928). Наилучший баланс между компактностью и качеством показывает конфигурация **`groups=4`**: сокращение параметров в 3.8 раза и FLOPs в 3.8 раза при потере точности менее 2% относительно baseline.

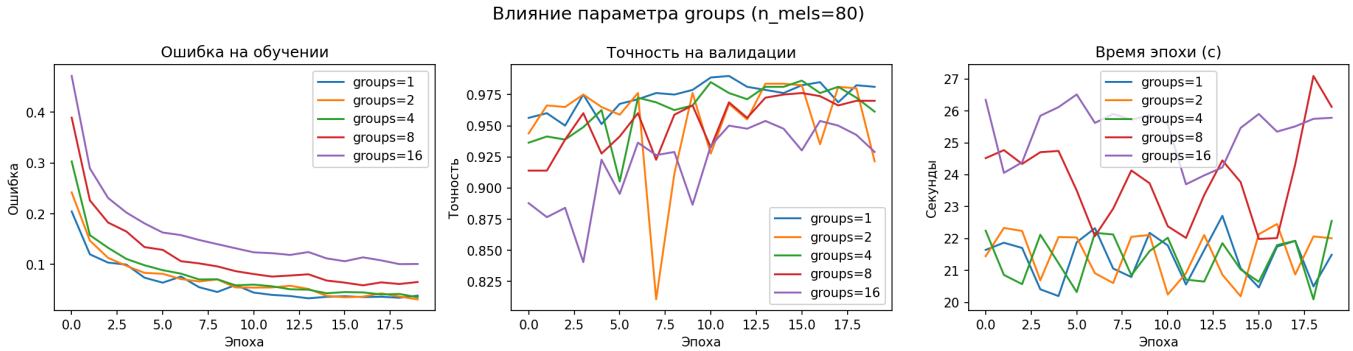

Рис. 4 — Кривые обучения для разных значений groups.

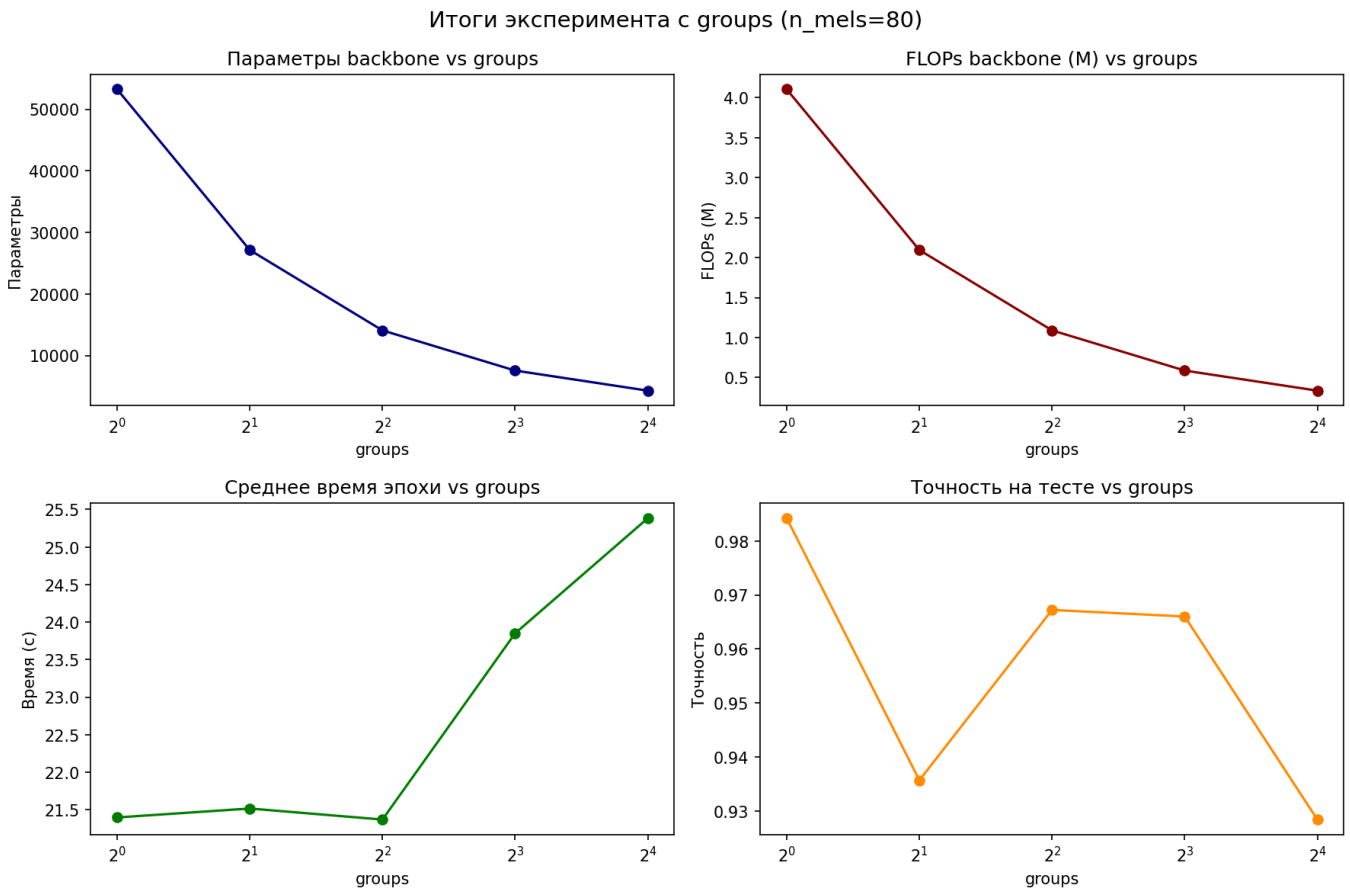

Рис. 5 — Зависимость параметров, FLOPs, времени эпохи и точности от groups.